IDS Pipeline

UNSW-NB15 — Network Intrusion Detection System

Data Cleaning	3 NaN cols filled	**Encoding**	OrdinalEncoder	**Log Transform**	36 skewed cols
2 Split Regimes	Random + Time	**Feature Eng.**	IP · Flow · Subnet	**3 Models**	XGB · LGB · RF
Threshold Tuning	Precision / Recall	**Hyperparameter**	RandomizedSearch	**Calibration**	Platt Scaling

Data Cleaning

What was wrong and exactly how it was fixed

	Problem	Fix	Why

`ct_ftp_cmd`
`ct_flw_http_mthd`
`is_ftp_login`
	NaN — but not missing	`Fill → 0`	NaN means no FTP/HTTP activity in that flow. Zero is semantically correct.

`attack_cat`	**"nan" stored as literal string**	`Replace → "Normal"`	Raw CSVs write the string "nan" for normal flows. Phase 15 multiclass needs a clean label.
`sport / dsport`	**Hex strings e.g. 0xC3B2**	`Cast → nullable Int32`	Nullable Int32 preserves NaN instead of silently coercing hex values to 0.
`All numeric cols`	**Default float64 — wastes RAM**	`Cast → float32 / int8`	float32 is sufficient precision for network stats. Halves memory: 2.5M rows × 49 cols.

Ordinal Encoding

Why OrdinalEncoder, not OneHotEncoder

Columns encoded

proto

TCP, UDP, ICMP, ... → integer

~130 unique values

state

FIN, INT, CON, ... → integer

~16 unique values

service

http, ftp, dns, ... → integer

~14 unique values

OrdinalEncoder — decision rationale

✓ **Tree models don't need OHE**

XGBoost, LightGBM, and Random Forest split on thresholds — integer codes work identically to one-hot columns.

✓ **Memory: 1 col vs 130 cols**

One-hot encoding proto (130 categories) would add 130 binary columns. Ordinal keeps it 1.

✓ **Unknown categories → -1**

handle_unknown='use_encoded_value' means new protocols at inference time map to -1 safely, no crash.

✓ **Fit on train only**

Encoder fitted on X_train, then applied to X_test. Test categories never influence the mapping.

Log Transform (log1p)

36 of 59 features transformed — why this matters

$$\log1p(x) = \log(1 + x)$$

Safe for zero values — no log(0) error

! Problem

Network features (sbytes, dur, flow_rate)

are extremely right-skewed.

A single 1GB transfer dominates the

gradient and overshadows thousands

of normal flows.

~ Effect of log1p

Compresses the extreme right tail.

Small values spread out, large values

are pulled inward.

Skewness > 1 → near-normal distribution

after transform.

✓ Result for the model

Gradient boosting learns faster.

No single feature dominates splits.

36 columns identified by skew > 1 threshold

on X_train only.

Same column list applied to X_test.

Two Split Regimes

Measuring best-case learning vs deployment reality

Random Split

How	Shuffled, stratified 80/20
Train	X_train_bal (SMOTE 50/50)
Test	X_test
Answer	How well can this model learn?
Score	Higher — best-case ceiling
Attack rate	~32% (preserved by stratify)

Time-Based Split

How	Sort by Stime, first 80% / last 20%
Train	X_tr_time_bal (SMOTE 50/50)
Test	X_te_time
Answer	How well will it deploy?
Score	Lower — realistic ceiling
Attack rate	~11% train / ~19% test

Both splits receive SMOTE — the only variable between them is the split strategy itself, not the class distribution

Feature Engineering

15 new features created from raw IPs and flow statistics

IP Topology Flags (×8)

src/dst_is_private

LAN vs internet traffic

src/dst_is_global

External-facing IPs

src/dst_is_multicast

Broadcast / multicast patterns

src/dst_version

IPv4 vs IPv6

Subnet (×2)

src_subnet

First 2 octets of source IPv4

dst_subnet

First 2 octets of destination IPv4

Scanning Behaviour (×1)

src_unique_dst

Unique destinations contacted by source

IP Frequency (×2)

src_freq

How often this source appears in training

dst_freq

How often this destination appears

Flow Statistics (×4)

duration_time Ltime − Stime = exact integer-second duration (more precise than raw 'dur' float)

Model Selection

Why these three — and what each contributes

XGBoost

Baseline

Parameters

```
n_estimators: 1000
learning_rate: 0.05
max_depth: 6
early_stopping: 30 rounds
subsample: 0.8
```

Depth-first tree growth. Gold standard for tabular data competitions. Strong regularisation via subsampling.

Slower on 3M+ rows. Both models hit 1000-tree ceiling → Phase 17 raises limit.

LightGBM

★ Chosen

Parameters

```
n_estimators: 1000
learning_rate: 0.05
num_leaves: 63
early_stopping: 30 rounds
colsample_bytree: 0.8
```

Leaf-wise growth. 3–5× faster than XGBoost on large datasets. Wins every metric on the time-based test.

ROC-AUC 0.9997 on time test. Gap vs random split: only 0.0001. Fewest false positives: 4,894.

Random Forest

Ensemble baseline

Parameters

```
n_estimators: 300
max_depth: 20 (capped)
max_features: sqrt
no early stopping
n_jobs: -1
```

Bagging of independent trees — different inductive bias from boosting. Robust to outliers. No hyperparameter sensitivity.

More memory. Slower inference. Used as a sanity check: if RF >> boosting, data has simple structure.

Threshold Tuning

Default 0.5 is rarely optimal for IDS — shift based on operational cost

The tradeoff

↓ Threshold (e.g. 0.3)

More attacks flagged

Cost: More false alarms

↑ Threshold (e.g. 0.7)

Fewer false alarms

Cost: More missed attacks

Optimal threshold

Maximise F1 (Attack)

Cost: Balance P and R

How to find it

1. **Get predicted probabilities**

```
model.predict_proba(X_val)[:, 1]
```

2. **Sweep thresholds 0.1 → 0.9**

```
for t in np.arange(0.1, 0.9, 0.01)
```

3. **Score each threshold**

```
f1_score(y_val, probs >= t, pos_label=1)
```

4. **Pick max F1 threshold**

```
best_t = thresholds[np.argmax(f1_scores)]
```

5. **Apply to test set**

```
y_pred = (y_prob >= best_t).astype(int)
```

Hyperparameter Tuning — RandomizedSearchCV

30 trials × 5-fold CV on 20% subsample, then re-train on full data

RandomizedSearch vs GridSearch

GridSearch: 5 params × 4 values = 1,024 fits | RandomizedSearch: 30 fits | Near-optimal in far fewer trials (Bergstra & Bengio 2012)

20% subsample for speed

5-fold CV on 3.5M rows ≈ hours
20% subsample ≈ minutes
Best params → re-train on 100% data

LightGBM search space

`n_estimators`	`randint(200, 1200)`
`learning_rate`	`uniform(0.01, 0.15)`
`num_leaves`	`randint(31, 256)`
`max_depth`	`randint(5, 20)`
`subsample`	`uniform(0.6, 0.4)`
`min_child_samples`	`randint(10, 100)`
`reg_alpha`	`uniform(0, 1)` ← L1
`reg_lambda`	`uniform(0, 1)` ← L2

Tuning workflow

1. **Read metrics_summary.csv**
   Auto-select best time-test model
2. **20% stratified subsample**
   Search on representative subset
3. **30-trial RandomizedSearchCV**
   Optimise ROC-AUC, 5-fold CV
4. **Re-train on full data**
   Best params + 100% training rows
5. **Evaluate + compare**
   Delta table vs baseline
6. **Save artifacts**
   tuned model + cv_results.csv

Probability Calibration

Why raw model probabilities are misleading — and how to fix them

The problem

Gradient boosting models are trained to minimise logloss — not to output calibrated probabilities. A model that says p=0.9 doesn't mean '90% chance of attack'. In practice, boosting models tend to push probabilities towards 0 and 1 (overconfident) or cluster them in the middle. This makes the raw probability score unreliable for risk scoring, alert prioritisation, or setting custom thresholds based on operational cost.

Platt Scaling (recommended)

What	Fit a logistic regression on top of the model's raw output scores using a held-out calibration set.
Input	Raw decision function scores (not probabilities)
Output	Calibrated probabilities that match actual frequency
Code	`CalibratedClassifierCV(model, method='sigmoid', cv='prefit')`
When	After training, before saving for inference

Why calibrate for IDS inference

✓ Risk scoring

p=0.92 should mean 92% of flows with this score are attacks — enabling tiered alerting.

✓ Custom threshold reliability

Threshold tuning only works correctly if the probability scale is meaningful.

✓ Alert prioritisation

SOC teams can sort alerts by calibrated probability and tackle highest-confidence first.

! Use a separate calibration set

Never calibrate on the training set. Use a fresh hold-out split to avoid over-fitting the calibration.

Results & Chosen Model

All three models tested — LightGBM time-split selected for deployment

Model	Split	ROC-AUC	Accuracy	Prec (Atk)	Recall (Atk)	F1 (Atk)	Cross-eval
XGBoost	Random	0.9998	99.44%	0.9751	0.9811	0.9781	0.9994
LightGBM	Random	0.9998	99.52%	0.9804	0.9815	0.9809	0.9997
Random Forest	Random	0.9998	99.40%	0.9672	0.9864	0.9767	0.9998
XGBoost	Time	0.9995	98.77%	0.9553	0.9830	0.9689	—
LightGBM ★	Time	0.9995	98.80%	0.9583	0.9810	0.9695	—
Random Forest	Time	0.9990	98.77%	0.9507	0.9882	0.9691	—

Generalisation gap (random – time ROC-AUC)

XGBoost	+0.0003	
LightGBM	+0.0003	near zero
Random Forest	+0.0008	largest

★ Chosen: LightGBM (time-split)

Saved as: `lgb_binary_time.txt`
Also: `CHOSEN_MODEL_lgb_time.txt`
Why: Tied best time-test ROC-AUC, smallest gap

- ✓ All three models achieve 98.77-98.80% accuracy on time test — deployment-ready performance
- ✓ Random Forest wins cross-eval (0.9998) but has largest gap (+0.0008) — relies more on time patterns
- ✓ LightGBM and XGBoost tied for smallest gap (+0.0003) — best temporal generalisation
- ! All models hit 1000-tree ceiling without early stopping — Phase 17 tuning raises n_estimators to 1200+